

1. Riassunto Esecutivo

Proton è un chatbot AI avanzato dedicato a supportare studenti universitari, liceali o di qualsiasi percorso formativo. Utilizza **AWS** come infrastruttura cloud e si basa sul framework **LangChain** per orchestrare il processo di *Retrieval-Augmented Generation (RAG)* e *Context-Augmented Generation (CAG)*. Gli utenti possono caricare documenti (PDF, appunti digitali, immagini di appunti scritti a mano). Le immagini vengono elaborate con **AWS Textract** per l'estrazione di testo. I documenti caricati sono salvati in **Amazon S3** e poi suddivisi in chunk semantici con LangChain. I chunk vengono indicizzati in **ChromaDB** (database vettoriale open source) utilizzando embedding di testo generati da **Amazon Titan Text Embeddings V2** (modello Bedrock). Il modello di linguaggio principale è **Anthropic Claude** (via Amazon Bedrock), scelto per le sue capacità di comprensione semantica, finestra di contesto estesa (fino a 200k–1M token **【2†L66-L74】**) e robustezza su compiti conversazionali complessi. L'architettura RAG conversational (con memoria chat) permette di mantenere il contesto delle conversazioni e generare risposte, quiz e simulazioni d'esame pertinenti al materiale caricato. L'interfaccia utente è basata su **Streamlit**, con autenticazione utenti tramite AWS Cognito. L'intera applicazione gira in container Docker orchestrati con **AWS Fargate/ECS** per semplificare il deployment e la scalabilità (senza gestione di server). Le credenziali e gli artefatti sensibili sono protetti con **AWS Secrets Manager** e **IAM**.

2. Obiettivi di Business

- **Educazione personalizzata:** fornire agli studenti un tutor virtuale capace di comprendere i propri appunti e materiale di studio, migliorando l'apprendimento personalizzato.
- **Efficienza nello studio:** automatizzare la creazione di quiz, esercizi ed esami simulati basati sul materiale del corso, riducendo il carico di lavoro degli insegnanti e migliorando la preparazione degli studenti.
- **Accessibilità del materiale:** centralizzare risorse didattiche (testi, note, immagini scansionate) in un knowledge base ricercabile, garantendo che l'IA abbia sempre accesso alle fonti più aggiornate fornite dallo studente.
- **Interazione naturale:** abilitare una conversazione naturale in linguaggio comune (Italiano, Inglese, ecc.) fra studente e chatbot, dove il chatbot risponde in maniera colloquiale, contestualmente alle informazioni apprese.
- **Espandibilità:** costruire un'architettura modulare su AWS che può crescere con il numero di utenti e documenti, consentendo l'integrazione futura di funzionalità avanzate (agenti multipli, interfacce vocali, pianificatori di studio, ecc.).

3. Requisiti Funzionali

- **Caricamento documenti:** l'utente può inviare file PDF, documenti di testo o immagini di appunti. I file vengono salvati su Amazon S3 e passati ad un processo di pre-elaborazione (OCR per le immagini con AWS Textract).
- **Elaborazione OCR:** le immagini di appunti vengono elaborate da AWS Textract per estrarne il testo in modo accurato, consentendo al sistema di trattarle come normali documenti testuali.
- **Gestione file e versioni:** i documenti caricati sono organizzati per utente e per corso, con metadati associati (es. nome corso, argomento). Eventuali aggiornamenti di un documento devono aggiornare la knowledge base.
- **Conversazione interattiva (Chatbot):** l'interfaccia principale è una chat in tempo reale (via Streamlit), dove lo studente pone domande o chiede esercizi, e il chatbot risponde sfruttando il knowledge base personalizzato.
- **RAG con memoria:** il sistema utilizza tecniche RAG conversazionali con memoria chat, per tenere conto del contesto delle domande successive. La cronologia delle conversazioni viene salvata nel database (storage persistente) per sessioni future.
- **Generazione di contenuti didattici:** il chatbot è capace di creare:
 - quiz a risposte multiple
 - domande aperte con verifica automatica
 - esercizi di ragionamento (es. problemi passo-passo)
 - simulazioni di esame (combinazioni di quiz e domande aperte)
- **Personalizzazione:** il chatbot tiene conto del materiale caricato dall'utente. Esso risponde principalmente sulla base di questi documenti, evitando informazioni esterne non pertinenti.
- **Autenticazione utenti:** accesso all'app tramite login (AWS Cognito). Ogni utente ha uno spazio privato dove vengono conservati i propri documenti, la chat history e i modelli di apprendimento.
- **Integrazione LangChain:** utilizzo di LangChain per orchestrare il caricamento dei documenti, il chunking semantico, la memorizzazione dei vettori in ChromaDB e l'interazione con Claude via Bedrock.
- **Archiviazione persistente:** la knowledge base vettoriale (ChromaDB) e la cronologia chat sono persistenti, in modo che il sistema ricordi dati e conversazioni al riavvio.
- **UI streamlit:** un'interfaccia web semplice basata su Streamlit con pagine dedicate a login, caricamento documenti, chat, cronologia, gestione corsi e dashboard di progresso.

4. Requisiti Non Funzionali

- **Scalabilità:** l'architettura su AWS deve scalare orizzontalmente al crescere del numero di utenti e documenti. L'uso di servizi serverless (Fargate, Bedrock) riduce la gestione di infrastruttura e permette di adattarsi automaticamente al carico (ad es. Avvio di più task Fargate su richiesta).
- **Disponibilità e Affidabilità:** i servizi scelti (AWS Fargate, Amazon Bedrock, S3) garantiscono elevata disponibilità e tolleranza ai guasti. Adottiamo best practice AWS per replica delle risorse (es. regioni multiple se necessario) e backup regolari dei dati su S3.
- **Manutenibilità:** il codice è organizzato in moduli chiari con documentazione (README, docstring, test). L'uso di container Docker e CI/CD semplifica il rilascio di nuove versioni. Ogni decisione architetturale è motivata e documentata per futura revisione.
- **Prestazioni:** la latenza di risposta del chatbot deve essere ragionevole (target < 2-3 secondi per risposte generiche, benché alcune query complesse con RAG possano richiedere più tempo). Per ridurre la latenza, utilizziamo endpoint Bedrock a bassa latenza per Claude e preferiamo embeddings generati in batch o configurazioni ottimizzate.
- **Sicurezza:** i dati sensibili (chiavi API, credenziali Cognito) sono gestiti con AWS Secrets Manager e accesso limitato da IAM. Le comunicazioni sono cifrate (TLS) end-to-end. S3 abilita la cifratura server-side standard. Non è necessaria cifratura aggiuntiva per GDPR, ma AWS gestisce comunque cifratura a riposo per default **[41†L14-L22]** .
- **Costo:** i servizi scelti sono bilanciati tra performance e budget. Ad esempio, ChromaDB open-source riduce costi di licenza, mentre Bedrock consente accesso a modelli avanzati con modelli pay-per-use. Le opzioni di deployment (Fargate) eliminano costi di gestione server.

5. Architettura di Alto Livello

L'architettura generale di Proton è basata sul modello RAG conversazionale. Un utente interagisce con l'interfaccia Streamlit (client web) per caricare file e chattare. I file vengono passati ad un microservizio in Python (girante su Docker) che li salva su S3 e avvia pipeline di elaborazione: per le immagini, chiama AWS Textract per OCR; successivamente, converte tutto in testo e invia i documenti a LangChain per il chunking semantico. I chunk risultanti sono indicizzati in ChromaDB come vettori embedding (frutto di modelli Titan Text Embeddings). Quando l'utente formula una domanda via chat, il backend recupera dalla memoria chat il contesto recente, genera query semantica (o vari query multipli), interroga ChromaDB per recuperare i chunk più rilevanti, e infine invia il prompt combinato (domanda + contesto + chunks) a Claude tramite Amazon Bedrock. Claude elabora la risposta e la restituisce, che viene mostrata nell'interfaccia. Tutte le operazioni di service-

to-service avvengono in VPC AWS privata, con componenti container orchestrati da ECS/Fargate. Il flusso dati è illustrato di seguito:

- **Utente** → [Streamlit UI] → invio documento / domanda
- **Upload doc** → [Backend Python] → S3 (salvataggio)
- **OCR (immagini)** → [AWS Textract] → testo estratto
- **Chunking semantico** → [LangChain] → generazione chunk
- **Embedding** → [Bedrock Titan] → vettori + **ChromaDB**
- **Interrogazione RAG** → [ChromaDB & LangChain] → contesto di conoscenza (chunks)
- **Chiamata LLM** → [Bedrock Claude] → generazione risposta
- **Mostra risposta** → [Streamlit UI]

Questa architettura garantisce che le risposte siano “grounded” nei documenti caricati dallo studente (Riducendo l’allucinazione 【13†L51-L60】), mantenendo al contempo un’interazione fluida grazie alla memoria conversazionale.

6. Architettura AWS

Proton è progettato come architettura cloud-native su AWS, sfruttando i seguenti servizi:

- **Amazon S3:** archivio oggetti per tutti i file caricati (documenti, appunti scansionati, immagini). Si abilita la cifratura a riposo automatica. È scalabile e gestisce versionamento dei documenti.
- **AWS Textract:** servizio per estrazione testo da immagini. È invocato dal backend e legge i file da S3 (integrazione nativa S3→Textract).
- **Amazon Bedrock:** fornisce i modelli LLM di Anthropic Claude (inclusi i modelli multi-modal se necessario) e i modelli di embedding Titan. Utilizziamo endpoint on-demand di Bedrock per generare embedding dei chunk e per interrogare Claude.
- **Amazon SageMaker (Notebook):** ambiente di sviluppo per prototipazione. Connesso al repo GitHub, facilita la sperimentazione (anche per eventuali training futuri). Non necessario a runtime, ma utile per data wrangling.
- **Amazon ECS / AWS Fargate:** orchestrazione dei container Docker. Ogni componente backend (API, worker LangChain, ecc.) gira in container. Fargate automatizza provisioning di CPU/RAM senza gestire server, ed è ideale per servizi containerizzati a richiesta 【36†L59-L68】.
- **AWS Cognito:** gestione utenti e autenticazione. Un **User Pool** AWS Cognito permette login/signup sicuri, con possibilità di MFA. Cognito gestisce credenziali, token e integrazione con federated login se richiesto.
- **AWS Secrets Manager:** conservazione sicura di segreti (chiavi API Bedrock, chiavi AWS, segreti OAuth, ecc.). Automatizza la rotazione e l’accesso controllato tramite IAM.

- **AWS CloudWatch**: logging e monitoraggio. Tutti i container inviano log applicativi su CloudWatch Logs. Metriche personalizzate possono essere tracciate con CloudWatch Metrics (es. latenza LLM, errori).
- **AWS IAM**: definizione dei ruoli e permessi. Esempio: ruolo per ECS/Fargate con accesso S3/Texttract/Bedrock minimi necessari. Ruoli separati per codice backend e per eventuali funzioni Lambda (non utilizzate qui).
- **Altri servizi** (se necessari):
- **AWS Certificate Manager (ACM)** per certificati SSL/TLS (Streamlit su HTTPS, eventuale CloudFront).
- **Amazon Route 53 / CloudFront** (opzionali) per distribuire l'interfaccia web con CDN e dominio personalizzato.
- **Amazon DynamoDB** (non previsto ora, ma potrebbe servire per chat memory a bassa latenza se non in Chroma).
- **Amazon CloudTrail** per audit log (tutte le chiamate API AWS sono registrate).

Le scelte chiave sono **Cognito** per l'autenticazione (in quanto fornisce un flusso utente completo senza gestire database password) e **Secrets Manager** per proteggere chiavi e token. **CloudWatch** garantisce la visibilità operativa, mentre **IAM** definisce chiaramente i permessi («principio del privilegio minimo»). Servizi non utili per questo progetto (e che quindi non verranno implementati) sono Amazon RDS (non necessario perché usiamo un vector store dedicato) e AWS Lambda (poiché il carico prevede processi lunghi e containerizzati anziché funzioni brevi).

7. Struttura del Repository

La repository GitHub di Proton sarà organizzata in modo modulare e chiaro, seguendo best practice Python e di sviluppo web. Un possibile layout è:

```
/proton-chatbot
├── README.md           # Documentazione di alto livello
├── LICENSE             # Licenza MIT
├── requirements.txt    # Dipendenze Python del progetto
├── .gitignore          # File da ignorare da git
├── /frontend           # Codice dell'interfaccia Streamlit
│   ├── app.py          # File principale Streamlit (UI)
│   ├── utils.py        # Utility specifiche UI
│   └── assets/         # Risorse statiche (CSS, immagini)
├── /backend            # API e logica server
│   ├── api/            # Endpoint REST o logica di routing
│   │   ├── __init__.py
│   │   └── routes.py
│   └── services/       # Moduli di servizio (es. Texttract, S3,
Bedrock)
│   └── rag_pipeline/   # Logica RAG: chunking, retrieval, query
```

generation	
├── auth/	# Gestione autenticazione con Cognito
├── utils/	# Funzioni ausiliarie generali
└── main.py	# Entry-point del server / worker
├── /rag	# Componenti LangChain specifici
│ ├── chunkers/	# Script per il chunking (semantic, recursive,
│ └── ecc.)	
│ ├── embedders/	# Configurazioni degli embedding (Titan, ecc.)
│ └── retrievers/	# Integrazione con ChromaDB
└── /models	# Configurazioni modelli LLM e embedding
(prompt templates, token limits)	
├── /docker	# File Docker
│ ├── Dockerfile.backend	# Dockerfile per il backend
│ ├── Dockerfile.frontend	# Dockerfile per la UI
│ └── docker-compose.yml	# (eventuale) orchestrazione multi-container
├── /scripts	# Script di utilità (deploy, setup, ecc.)
├── /tests	# Test automatizzati
│ ├── unit/	# Unit tests di singole funzioni/metodi
│ └── integration/	# Test end-to-end (pipeline RAG, API)
└── /docs	# Documentazione di progetto estesa
│ ├── architecture.md	# Dettagli architetturali
│ └── design_decisions.md	# Motivazioni delle scelte progettuali

Motivazione: una struttura simile, ispirata alle linee guida di Python e grandi progetti open-source [【34†L53-L62】](#) [【34†L139-L147】](#) , separa nettamente l'interfaccia (frontend), il core applicativo (backend), la pipeline RAG (rag), e le configurazioni. Ogni cartella include un `__init__.py` se necessario. I servizi AWS (S3, Textract, etc.) sono astratti in moduli dedicati (`services/`), mentre il routing delle chiamate API (se esiste una API REST) è in `api/routes.py`. I modelli LLM e gli schemi di prompt si trovano in `/models` per facilità di modifica. La cartella `/docker` contiene i Dockerfile e uno script `docker-compose` (utile per sviluppo locale). I test sono suddivisi in unitari e di integrazione, come consigliato [【34†L130-L139】](#) . In tal modo, il repository rimane ordinato, facile da navigare e pronto per contributi di sviluppo.

8. Flusso dei Dati

Il flusso dati nel sistema segue questi passi:

1. **Upload Utente:** lo studente carica un documento tramite l'interfaccia Streamlit. Il file viene inviato al backend (es. via chiamata API HTTP).

2. **Archiviazione S3:** il backend salva il file su un bucket S3 dedicato (ad esempio proton-docs). S3 gestisce la persistenza e l'accesso sicuro (possiede permessi IAM per accettare i file).
3. **Elaborazione OCR (se applicabile):** se il file è un'immagine, il servizio backend richiama AWS Textract (API StartDocumentTextDetection o simili) puntando all'oggetto S3, ottenendo il testo riconosciuto. Questo testo viene quindi combinato con eventuali altri file testuali.
4. **Chunking Documenti:** il testo completo di ogni documento viene inviato a LangChain, che applica *semantic chunking*. Il testo è suddiviso in blocchi semantici significativi (per es. paragrafi o gruppi di frasi correlate, anziché frasi fisse) **【23†L119-L127】**. Ciò garantisce che ogni chunk abbia coerenza semantica interna. In pratica, LangChain può usare un embedders di frasi per raggruppare e tagliare i testi.
5. **Creazione Embedding:** ogni chunk di testo viene passato al modello embedding **Amazon Titan Text Embeddings V2** (amazon.titan-embed-text-v2:0), tramite Bedrock. Titan V2 supporta fino a 8192 token di input e produce vettori a 1024 dimensioni **【16†L25-L33】**, adatti per compiti di retrieval. Il risultato è un vettore semantico per ogni chunk.
6. **Salvataggio nel Database Vettoriale:** i vettori generati vengono memorizzati in **ChromaDB**, insieme a metadati (es. riferimento documento, posizione all'interno del documento, utente, ecc.). ChromaDB indicizza internamente i vettori (utilizzando SQLite o object store) per operazioni di ricerca simile velocizzata **【32†L579-L588】**.
7. **Richiesta di Informazione (Retrieval):** quando lo studente invia una domanda nella chat, il backend costruisce un vettore di query (o più varianti di query, in caso di multi-query RAG) usando lo stesso modello embedding. ChromaDB viene interrogato con questo vettore per restituire i chunk più rilevanti (i "vicini" nello spazio vettoriale).
8. **Memoria Conversazionale:** oltre ai chunk ricavati dal knowledge base, il sistema include il contesto recente della conversazione (precedenti domande/risposte) come parte del prompt, in modo da creare un RAG *conversazionale*. LangChain può recuperare queste memorie dall'archivio chat (es. DynamoDB, o un semplice database) e includerle.
9. **Interrogazione LLM:** i chunk rilevanti + la domanda + il contesto chat sono concatenati per formare il prompt finale. Questo prompt viene inviato ad Amazon Bedrock utilizzando il modello Claude selezionato. Claude elabora il prompt e genera la risposta desiderata.
10. **Output Utente:** la risposta testuale (e.g. soluzione di esercizio, feedback alla risposta aperta) viene inviata al frontend Streamlit che la mostra allo studente. La conversazione viene salvata nel database chat memory per uso futuro.

11. **Ciclo di Feedback:** eventuali chiarimenti o domande successive del collegano diventano nuovi input per LangChain, con il flusso che si ripete dal punto 7, utilizzando la memoria conversazionale per contestualizzare.

In questo flusso, notiamo che **S3** è usato solo per il salvataggio grezzo dei file di input, mentre ChromaDB mantiene solo vettori (non la versione testuale completa). Non c'è conflitto: S3 conserva i dati originali, ChromaDB contiene le rappresentazioni derivate. In pratica, si usa S3 per **archiviazione dati** e ChromaDB come **indice di ricerca vettoriale**. Anche se Amazon sta sviluppando “S3 Vectors” per funzioni simili, attualmente l'uso combinato S3 (per file) + Chroma (per ricerca semantica) è una soluzione collaudata.

9. Architettura RAG

Proton sfrutta una pipeline RAG *conversazionale*. Analizziamo le possibili varianti di RAG e motiviamo la scelta:

- **RAG Base (Simple/Basic RAG):** prevedrebbe una singola query embed, ricerca dei documenti più simili e generazione della risposta. È facile da implementare e veloce [4†L75-L82]. Tuttavia, la RAG di base non gestisce lo stato conversazionale e si basa su un unico recupero, il che la rende adatta solo a Q&A puntuali. Inoltre, *Basic RAG* può soffrire di imprecisioni su domande complesse o che richiedono più fonti [7†L202-L210]. Per un uso iniziale e corpus piccolo, il RAG base può andare bene, ma non risponde a esigenze di chat continuativa.
- **RAG Conversazionale (RAG con memoria):** integra la memoria della conversazione. Ad esempio, il sistema memorizza le domande e risposte precedenti e le include come contesto nelle query successive [4†L98-L105] [4†L108-L116]. In pratica, LangChain tiene uno *chat history memory* che influenza il recupero. Questo approccio è ideale per un tutor virtuale: lo studente può porre domande di follow-up e il chatbot “ricorda” il contesto [4†L108-L116]. La *Conversational RAG* riduce ripetizioni inutili e permette interazioni più umane [4†L118-L124].
- **Agentic RAG:** prevede che l'AI agisca come un “ricercatore” autonomo, decida multipli passi di ricerca, piani di azione e strumenti da usare [4†L130-L139]. Ha vantaggi su domande multi-step e ricerche approfondite, ma richiede molti chiamate ridondanti e complessità operativa [4†L150-L158]. Per Proton (caso educativo), questa complessità appare eccessiva. Non serve un agente che genera piani di ricerca autonomamente; il contesto è limitato ai materiali forniti dallo studente. L'Agentic RAG ha costi computazionali maggiori e latenza più alta [4†L150-L158], per cui non è scelto.
- **RAG Ibrido (Hybrid Search RAG):** combina ricerca semantica (embeddings) e ricerca a parole chiave [11†L64-L73]. Ciò migliora precisione, in particolare su termini specifici (spesso utili in domini tecnici). Tuttavia, richiede la configurazione di due sistemi (un motore BM25 o simile in parallelo). Per un uso didattico generico,

questo può essere superfluo. I benefici di un RAG ibrido (maggiore accuratezza sui sinonimi e robustezza) sono reali 【11†L64-L73】 , ma il progetto può iniziare con sola ricerca semantica e valutare l'ibridazione in futuro se emergessero problemi di accuratezza.

- **Multi-Query RAG:** il sistema genera più variazioni della domanda (con LLM) e fa ricerche multiple, unendo i risultati 【9†L223-L231】 . Questo recupera documenti pertinenti che un singolo phrasing potrebbe perdere. I vantaggi sono evidenti nel superare gap linguistici tra linguaggio utente e documentazione 【9†L223-L231】 . Tuttavia, implementare Multi-Query RAG aumenta calcoli (più embedding e ricerche) e complessità di fusione dei risultati. Per un MVP iniziale, ci concentriamo su un singolo vettore di query. Multi-Query può essere aggiunto in una fase successiva se serve maggiore recall.
- **Conclusione:** la strategia prescelta è la *Conversational RAG* di base con possibilità di evolvere verso ibrido o multi-query se necessario. Questa scelta si basa sulla necessità di mantenere stato conversazionale e coerenza, senza la complessità di agenti autonomi o ricerche ibride fin da subito. Le alternative non adottate (Agentic, Hybrid, Multi-Query completi) sono ritenute sovradimensionate per la fase iniziale: l'attenzione è sulla qualità delle risposte educative e l'esperienza chat fluida, non su ricerca multi-step o database eterogenei 【7†L202-L210】 【4†L150-L158】 .

Pipeline RAG dettagliata: l'utente invia una domanda; il sistema la trasforma in un embedding e interroga ChromaDB; recupera i top-N chunk (es. N=3–5) più rilevanti; combina questi chunk con la domanda e con la memoria della chat; costruisce un prompt per Claude; invia il prompt a Bedrock e riceve la risposta generata; mostra la risposta. Eventualmente, il sistema può includere *top-p* di Claude per generare più varianti di risposta ed eseguire filtri interni. In pratica, LangChain coordina il flusso: RetrievalQA con memory, dove il retriever è Chroma e il generatore è Claude. Questo garantisce risposte “grounded” e coerenti con i materiali forniti dallo studente 【13†L51-L60】 【4†L75-L82】 .

10. Architettura CAG

La *Context-Augmented Generation (CAG)* è una tecnica emergente simile al RAG ma che sfrutta direttamente l'estensione di contesto dei moderni LLM. Ecco una panoramica:

- **Definizione di CAG:** CAG carica documenti interi nel contesto del modello senza usare un sistema di recupero separato 【14†L46-L54】 . In pratica, l'LLM vede direttamente tutto il contenuto rilevante e genera risposte basate su quel contesto completo. Questo è diventato praticabile grazie all'aumento delle finestre di contesto (oggi fino a 100k–200k token nei modelli più recenti, e persino fino a 1M token in preview 【2†L66-L74】 【14†L46-L54】).
- **Quando usare CAG:** è indicato quando i documenti di interesse sono di dimensione limitata (sotto 1-2M token totali) 【14†L98-L101】 e si desidera la

massima accuratezza senza preoccuparsi di retrieval. Ad esempio, se lo studente ha solo pochi appunti brevi, potremmo passare interi documenti a Claude. CAG elimina la complessità del chunking e della gestione del vector store 【14†L73-L82】, risultando in un flusso più semplice. Inoltre, con modelli come Claude 4 Sonnet (context 200k) o futuri con 1M token 【2†L66-L74】, caricare documenti interi è fattibile.

- **Differenze RAG vs CAG:** RAG richiede un'infrastruttura di embedding e DB (ChromaDB) 【14†L73-L82】. CAG no: basta inviare direttamente i bytes del file e il prompt al modello 【14†L46-L54】. Tuttavia, CAG richiede modelli con contesti molto ampi e può richiedere costi di token più alti (si paga per tutti i token di documento caricati). RAG scalda meglio con grandi quantità di dati, poiché recupera solo i chunk rilevanti 【14†L73-L82】. Ad esempio, per un libro di 500 pagine, CAG richiederebbe un modello con finestra >100k token, mentre RAG utilizza chunk più piccoli.
- **Integrazione in Proton:** teoricamente, si potrebbe realizzare una modalità CAG caricando interi PDF in Claude tramite Bedrock (usando la API di generazione che accetta documenti) 【14†L46-L54】. Però, dato che Proton gestisce decine di documenti per utente, preferiamo l'approccio RAG. Permette di aggiungere nuovi documenti incrementali e interrogare solo parti pertinenti. CAG può essere considerato in futuro per casi d'uso specifici (es. domanda che richiede contesto completo di un documento piccolo).
- **Conclusione:** mantenere CAG come opzione sperimentale. Per la fase iniziale, la pipeline rimane su RAG con chunking. CAG si differenzia principalmente nel flusso (nessuna fase di retrieval) 【14†L73-L82】 e nella necessità di modelli con contesti estremamente lunghi. Verrà valutato successivamente se utilizzare CAG in scenari dove il contesto (es. un singolo PDF di poche pagine) rende praticabile questa tecnica.

11. Strategia di Embedding

Il sistema usa embedding di testo per indicizzare semanticamente i chunk. **Scelta modello:** utilizziamo **Amazon Titan Text Embeddings V2** (amazon.titan-embed-text-v2:0) offerto da Bedrock 【16†L25-L33】. Titan v2 è progettato per compiti di retrieval testuale e supporta input fino a 8192 token (~50k caratteri) 【16†L25-L33】, consentendo chunk di dimensioni maggiori rispetto a molte altre opzioni. Produce vettori di dimensione 1024 (di default) con alta qualità semantica. Inoltre, Titan v2 ha robuste capacità multilingue e di dominio specifico 【18†L171-L179】, rendendolo adatto a materiali didattici anche misti (italiano, inglese, diagrammi testuali). Il modello è integrato in Bedrock, semplificando l'infrastruttura (non serve gestire un endpoint separato) e garantendo scalabilità e sicurezza AWS.

Alternative valutate:

- *OpenAI text-embedding-3-large (o ADA)*: modello molto performante per compiti generali e con buone prestazioni semantiche 【18†L131-L139】. Tuttavia, supporta solo ~4096 token e non è nativamente integrato in AWS. Utilizzarlo avrebbe richiesto chiamate esterne o implementare embedding custom. Inoltre, Titan V2 offre vector di dimensioni riducibili (512 o 256) con selezione automatica nella richiesta, offrendo flessibilità.
- *Cohere embed-english-v3*: eccelle in contesti open-domain 【18†L69-L78】, ma supporta solo 2048 caratteri (512 token) e non offre il livello di integrazione AWS che Titan ha. Non scelto per i limiti di contesto e per essere meno conveniente se si vogliono rimanere entro AWS Bedrock.
- *modelli locali (FAISS, Hugging Face)*: possibile usare ad esempio sentence-transformers/all-mpnet-base-v2 o simili con una libreria on-premise. Ciò ridurrebbe i costi di Bedrock, ma richiederebbe potenza di calcolo self-managed (GPU CPU) e updating continuo. Inoltre, Titan V2 è disegnato per essere eseguito in AWS con ottimizzazioni.
- *AWS S3 Vector store (in preview)*: AWS sta introducendo S3 Vectors 【38†L1-L4】, ma al momento utilizziamo ChromaDB con embedding esterni.

Motivazioni scelte: Titan v2 offre il miglior bilanciamento fra qualità semantica, capacità di contesto e integrazione con il resto della piattaforma AWS. È anche scalabile via endpoint Bedrock (on-demand e throughput) 【16†L25-L33】. Coherentemente con la nostra architettura, utilizziamo embedding su chunk non troppo grandi (anche se Titan supporta fino a 8k token). In futuro, potremo sperimentare altezze minori di vettori (512 dim) per ridurre storage/latency, se necessario. Complessivamente, Titan v2 è allineato all'utilizzo RAG su dati aziendali/didattici 【18†L171-L179】.

12. Strategia di Chunking

Per massimizzare la qualità del retrieval, Proton adotta il **Semantic Chunking**. Di seguito le motivazioni e il confronto con altre strategie:

- **Semantic Chunking**: come descritto, questa tecnica suddivide il testo in blocchi *semanticamente coesi*, dove ciascun chunk ha un significato completo. Si basa sugli embedding: si calcolano embed delle frasi e si raggruppano frasi affini 【23†L119-L127】. In questo modo i chunk risultano “interi” dal punto di vista del contenuto, il che migliora la precisione della ricerca 【23†L119-L127】. Ad esempio, frasi correlate saranno mantenute nello stesso chunk, evitando che un concetto venga spezzato in due parti indipendenti. Lo svantaggio è un overhead di calcolo maggiore rispetto al semplice splitting, ma garantisce risultati più pertinenti in una Q&A basata sul contenuto.
- **Fixed-size Chunking**: suddivide semplicemente per numero fisso di token (es. 1024 token per chunk) 【23†L93-L100】. È molto semplice da implementare e veloce 【23†L93-L100】. Tuttavia, non rispetta la struttura semantica del

documento: un chunk può interrompere una frase o un paragrafo importante. Proveremmo questa tecnica se fosse necessaria una baseline semplice, ma preferiamo semantic chunking per non penalizzare la precisione.

- **Recursive Chunking:** applica logiche iterative (es. per paragrafo, poi per frase) 【23†L101-L110】. È un buon compromesso tra fixed-size e contenuto logico, ma richiede parametri multipli (caratteri, frasi) e può diventare complesso. Non ci aspettiamo di doverlo usare, a meno che la semantic non dia problemi.
- **Document-specific Chunking:** sfrutta markup/struttura (es. paragrafi, sezioni) 【23†L111-L120】. Il nostro materiale di partenza è vario (note libere, PDF di lezioni); potremmo riconoscere titoli o capitoli, ma non ci basiamo su strutture formali. Pertanto, la semantic chunking, che è agnostica alla formattazione e orientata al contenuto, è più flessibile.
- **Token Chunking:** semplicemente conteggia token. Non adatto perché non considera il senso del testo e può creare chunk incoerenti.

Motivazione scelta: poiché Proton deve rispondere in modo accurato alle domande basandosi sulla coerenza dei concetti, è fondamentale che i chunk mantengano contesto completo 【23†L119-L127】. Ad esempio, in un testo di biologia, un concetto come “sintesi proteica” spesso implica più frasi: semantic chunking li tiene insieme. Questa scelta giustifica lo sforzo aggiuntivo. Le alternative (chunk fissi, ricorsivi) sono più facili, ma rischiano di “tagliare” argomenti a metà, riducendo l’efficacia del RAG. In sintesi, semantic chunking massimizza la qualità del retrieval 【23†L119-L127】 e quindi la pertinenza delle risposte.

13. Design del Database Vettoriale

Per conservare e interrogare i vettori embedding scegliamo **ChromaDB** come archivio vettoriale principale. Le motivazioni sono le seguenti:

- **Open-Source e Semplicità:** ChromaDB è open-source e orientato allo sviluppo rapido e prototipazione 【25†L150-L154】. Si integra facilmente in Python con SDK dedicati, cosa ideale per una start-up o progetto proof-of-concept. Non serve gestire un cluster complesso all’inizio; Chroma può essere eseguito come servizio leggero.
- **Funzionalità:** supporta query vettoriali efficienti, ricerche per somiglianza, e anche ricerca full-text e metadata 【32†L579-L588】. Ad esempio, Chroma permette di associare ai vettori etichette e filtri (anche se le query filtro avanzate sono più limitate rispetto a DB enterprise).
- **Flessibilità:** Chroma utilizza internamente SQLite o un object store, permettendo di memorizzare molti vettori senza infrastrutture dedicate 【32†L579-L588】. La documentazione è semplice e offre SDK per Python che si integrano con LangChain out-of-the-box.

- **Scalabilità iniziale:** per decine di documenti per utente, Chroma è più che sufficiente. Anche se manca di sharding automatico, la soluzione basterà fino a un volume moderato. In futuro, se i dati crescono molto, considereremo alternative gestite.

Confronto con altre soluzioni:

- **Pinecone:** è un database vettoriale fully-managed, con scaling orizzontale e ottimizzazioni industriali [25†L99-L102] . È ottimo per produzioni su larga scala e gestisce milioni di vettori con facilità. Tuttavia, è un servizio a pagamento esterno e richiede integrazione (API diversa). Per Proton, Pinecone avrebbe offerto alta affidabilità e semplicità d'uso in produzione, ma ai costi di un servizio gestito. Inoltre, Pinecone è “black box” e aggiungerebbe complessità nella gestione.
- **Weaviate:** supporta schemi complessi e ricerche ibride (inclusa integrazione con LLM) [25†L155-L164] . È open-source e offre motori di ricerca ibridi. Potremmo considerarla in futuro se desideriamo una gestione schematica dei dati. Ma la configurazione iniziale è più complessa di Chroma. Non la usiamo ora perché il nostro focus non è su ontologie complesse, ma su semplice retrieval dai documenti.
- **FAISS:** è una libreria C++/Python per indicizzazione vettoriale di Facebook. Non è un servizio standalone, ma un toolkit. Implica gestire manualmente i dati e l'hosting. FAISS offre prestazioni eccellenti su grandi dataset, ma richiede lavoro di engineering (e.g. saving/loading index). Per un progetto proo-of-concept, Chroma è più “plug and play”.
- **OpenSearch:** AWS OpenSearch ora supporta vettori (k-NN), quindi tecnicamente potrebbe fungere da DB vettoriale. Ma OpenSearch è pesante (gestione cluster Elasticsearch) e più orientato a ricerche full-text enterprise. Non scelto perché sovradimensionato per il nostro uso e più complicato da configurare.
- **pgvector:** estensione Postgres per vettori. Se già si usa PostgreSQL, è un'opzione integrata e open-source. Tuttavia, richiede un database Postgres attivo, che non vogliamo introdurre se non necessario. Per ora, non abbiamo bisogno della robustezza transazionale di un DB relazionale.
- **Cloudflare Vectorize, Qdrant, Milvus:** alternative recenti. Qdrant (open source) ha feature di filtering e scaling; Milvus è distribuito e scalabile. Queste soluzioni sono valide per carichi maggiori, ma anche più complesse da gestire. Non utilizzate al momento.

Perché Chroma è la scelta migliore: offre un rapido avvio (installation zero/config minimale), è gratuito per il carico iniziale e ben supportato da LangChain (SDK). Inoltre, in un contesto accademico/prototipo è consigliato come primo passo [25†L150-L154] . In una coda futura si potrà passare a soluzioni più robuste come Pinecone o Qdrant se necessario, ma per ora ChromaDB dà la “via più semplice” a fronte di buone performance. Come riportato da esperti, Chroma rappresenta un buon punto di partenza per prototipi RAG [25†L150-L154] [32†L576-L584] .

14. Autenticazione e Autorizzazione

L'app Proton richiede che solo utenti registrati possano accedere ai propri dati. Abbiamo scelto la soluzione AWS più immediata:

- **AWS Cognito (User Pool + Identity Pool):** imposta un User Pool per registrazione/login (supporta email/password, social login, MFA). Cognito gestisce la verifica email, token JWT e profiling utente. I front-end (Streamlit) ottengono token da Cognito al login. Un Identity Pool può quindi mappare il token a permessi AWS (es. permessi S3 per caricare documenti nel proprio bucket utente). Cognito è integrato con IAM, quindi possiamo definire policy per consentire agli utenti di accedere solo ai propri spazi. È una soluzione “serverless” e altamente scalabile per gestire l'autenticazione senza scrivere codice personalizzato.
- **Amazon IAM:** usato per definire i ruoli dei servizi. Per gli utenti, useremo role temporanei assegnati da Cognito (principio del privilegio minimo). Per i container e lambda, usiamo ruoli IAM con accesso S3/Texttract/Bedrock limitati a questi scopi. Ad esempio, il role di Fargate avrà i permessi `s3:PutObject`, `s3:GetObject` su bucket specifici, e `texttract:*` sul bucket di input.
- **AWS Secrets Manager:** protegge le credenziali dell'app (chiavi API Claude/Bedrock, configurazioni DB, segreti OAuth). All'avvio, i container caricano i segreti da Secrets Manager invece di hardcodarli o tenerli in file. Questo evita esposizione accidentale di credenziali e ne facilita la rotazione.
- **Autorizzazione:** utilizziamo controlli di accesso a più livelli. A livello app, Cognito garantisce che un utente possa vedere solo i propri file (filtrando le query S3 e Chroma per prefisso utente). A livello AWS, le policy IAM assicurano che un utente non possa effettuare operazioni fuori dal proprio scope. Ogni risorsa (bucket S3, namespace Chroma, ecc.) sarà isolata per utente/tenant.

Le alternative come **AWS Lambda** per l'autenticazione non sono necessarie (Cognito fa il suo lavoro). L'approccio Cognito+IAM è considerato best practice per applicazioni web AWS semplici e robuste. Non è implementata una crittografia aggiuntiva lato applicativo sui dati utenti, dato che i servizi AWS usati (S3, Secrets Manager) forniscono cifratura automaticamente. Se il requisito cambiassero, potremmo considerare il client-side encryption con KMS, ma ad oggi non serve.

15. Sicurezza

Sebbene Proton non necessiti di requisiti di sicurezza militare, seguiamo buone pratiche AWS:

- **TLS (Encryption in Transit):** tutte le comunicazioni tra client (browser) e Streamlit sono su HTTPS (usando certificato ACM su un Application Load Balancer, o

CloudFront). Anche le chiamate interne (front-end → backend) avvengono via HTTPS. Questo protegge i dati in transito.

- **Accesso alle risorse AWS:** come detto, usiamo IAM e Cognito per segregare dati. I bucket S3 sono privati, accessibili solo con credenziali valide. I ruoli Fargate hanno policy a “least privilege”.
- **Protezione credenziali:** nessuna API key è nel codice sorgente. Le credenziali Bedrock e altre risorse sensibili sono in Secrets Manager. Solo i ruoli autorizzati possono leggere quei segreti.
- **AWS KMS (Encryption at Rest):** S3 e Secrets Manager cifrano automaticamente i dati a riposo con chiavi gestite da AWS KMS 【41†L14-L22】. Quindi anche se non implementiamo cifratura manuale, i dati sui dischi AWS sono protetti.
- **Audit e Logging:** grazie a AWS CloudTrail, ogni chiamata API alla piattaforma (ad es. operazioni su S3, chiamate Textract/Bedrock, modifiche ai pool Cognito) è tracciata. I log delle API (eventuali errori e accessi) sono registrati in CloudWatch per analisi.
- **Protezione dati utente:** i dati personali (email, credenziali) sono sotto Cognito e KMS. Le conversazioni e i documenti didattici sono proprietà dell’utente; li isoliamo tramite permessi. Non è stato richiesto di memorizzare dati estremamente sensibili, quindi la semplice cifratura AWS a riposo è sufficiente 【41†L14-L22】.
- **Non uso di crittografia aggiuntiva:** come richiesto, non useremo crittografia lato applicazione sui dati in uso. Sfruttiamo le funzionalità AWS (S3 SSE, KMS) per semplificare. Se in futuro se ne dovesse avere la necessità, si può usare AWS KMS con client-side encryption, ma oggi non è necessario.

In sintesi, seguiamo le linee guida AWS sulla sicurezza, garantendo cifratura trasparente nei servizi e controlli di accesso con IAM/Cognito. Non introduciamo complessità non necessarie come VPN o WAF, poiché l’app si rivolge a studenti generalisti senza requisiti GDPR stringenti (oltre alle normali best practice). Ribadiamo che **AWS raccomanda l’uso di cifratura sia a riposo che in transito come controllo aggiuntivo** 【41†L14-L22】: la piattaforma lo offre di default.

16. Strategia di Deployment

Per il deployment in produzione, consideriamo le seguenti opzioni AWS:

- **EC2 (VM tradizionali):** comporta gestire istanze, patch, scaling manuale. Non serverless. Non ideale per il nostro scenario in cui vogliamo container facilmente orchestrati. Scartato per complessità di gestione e maggiore tempo di rilascio.
- **ECS (Elastic Container Service) + Fargate:** soluzione serverless per container. Deployiamo i Docker del backend e frontend come *task* ECS su Fargate. Non dobbiamo gestire i nodi: AWS scala CPU/RAM secondo configurazione. Perfetto per

carichi variabili e per l'ambiente multi-servizio (API, crawler, ecc.) 【36†L59-L68】. Fargate supporta anche l'uso di load balancer e l'integrazione IAM.

- **AWS Lambda:** funzioni serverless per compiti brevi. Non si adatta bene al nostro carico principale (Richieste LLM e pipeline di documenti non sono estremamente brevi e hanno bisogno di container con stickiness e memoria maggiore). Lambda limitato a 15 min di esecuzione 【36†L75-L82】 e massimo 10GB di memoria, e non si integra nativamente con carichi di lavoro containerizzati di grande complessità.
- **EKS (Kubernetes):** orchestratore Kubernetes AWS. Potente, ma introduce complessità notevole: gestione cluster, nodi, upgrade, costi significativi. Utile per organizzazioni grandi, ma eccessivo per una singola applicazione di medie dimensioni.

Scelta principale: ECS con Fargate. Motivazioni: offre infrastruttura serverless (riduce operazioni) mantenendo la flessibilità dei container 【36†L59-L68】. Supporta container Python con tutti i servizi AWS necessari (S3, Textract, Bedrock). Rispetto a Lambda, Fargate è progettato per *contenitori persistenti* e può gestire state temporanee e connessioni lunghe. Consente un'allocazione precisa di risorse CPU/RAM ad ogni task (fino a 120 GiB RAM e 16 vCPU 【36†L99-L102】), utile per Claude con carichi modesti. Utilizzando ECS con Fargate otteniamo deployment automatico (tramite ECS Task Definitions e GitHub Actions) e auto-scaling in base al numero di richieste. Questo approccio semplifica il monitoring (CloudWatch integrato) e garantisce alta disponibilità senza server dedicati.

Gli altri servizi (EKS/EC2) non sono stati adottati: **EC2** richiederebbe gestire cluster e OS, aumentando costi di manodopera; **Lambda** è limitato in durata e memoria, inadatto a un'app chat a stato; **EKS** è overspecializzato per scenari enterprise. Pertanto, la soluzione consigliata è **Docker su ECS-Fargate**, potenzialmente dietro a un ALB per instradare web (streamlit) e API. In futuro, se la scala cresce enormemente, potremmo riconsiderare servizi gestiti (ad es. EKS per microservizi complessi), ma inizialmente Fargate è ottimale.

17. Architettura Docker

Il progetto sarà containerizzato interamente per garantire portabilità e isolamento delle dipendenze. Descriviamo come:

- **Dockerfile:** il file Dockerfile principale (nella cartella /docker/Dockerfile.backend) installerà Python, le librerie di LangChain, AWS SDK (boto3), il client Bedrock, ChromaDB e Textract SDK. Esempio di passaggi:
- Partire da immagine base Python 3.10 slim.
- Copiare requirements.txt e installare dipendenze Python (pip install).
- Copiare il codice backend nel container.
- Esporre le porte necessarie (es. 8080 per API).

- Impostare il comando di avvio (es. `uvicorn main:app --host 0.0.0.0 --port 8080`).
- **Motivazioni containerizzazione:** facilita la configurazione degli ambienti (lo stesso container in locale, sviluppo e produzione). Le dipendenze (LangChain, Chroma, ecc.) sono complesse e immutabili tra ambienti; containerizzare garantisce coerenza. Con ECS-Fargate, il deployment è basato su immagini Docker (che vengono caricate su Amazon ECR). In produzione, useremo ECR come registro privato di immagini.
- **Integrazione Docker + AWS:** con ECS, il processo è: costruisci l'immagine localmente (o via CI/CD su GitHub Actions), push su ECR; poi aggiorni il task definition ECS per usare la nuova immagine. Fargate esegue i container dentro VPC.
- **Deployment del container:** AWS ECS gestisce la scalabilità orizzontale. Possiamo configurare servizi ECS per il backend e frontend, con target di CPU/RAM e regole di auto-scaling. Ad esempio, aumentare task backend se la latenza sale. Anche il repository Docker (/docker) avrà un `docker-compose.yml` per facilitare test in locale (simulando l'intera architettura multiprodotto).
- **Vantaggi operativi:** containerizzare permette isolamento (ogni servizio gira nel proprio container indipendente). La CI può eseguire test con la stessa immagine destinata a prod. La pipeline CD può fare rolling updates senza downtime. Inoltre, si riduce la dipendenza dalle configurazioni della macchina host.

L'uso di Docker in combinazione con AWS favorisce la mobilità del progetto. Ad esempio, potremmo spostare facilmente il carico da una VPC all'altra o tra account AWS semplicemente tirando l'immagine ECR. Si sfruttano anche i vantaggi di ECS: monitoring integrato, integrazione con IAM (permette a container di assumere ruoli) e con CloudWatch per i logs. In sintesi, Docker ci permette di impacchettare Proton come una serie di microservizi coerenti, semplificando lo sviluppo e il deployment in AWS.

18. Strategia di Testing

Per garantire la qualità del sistema Proton, implementeremo diversi livelli di test:

- **Unit Test:** test veloci e isolati per singole funzioni e moduli. Ad esempio, testare:
- Estrazione del testo dall'OCR (mockando Textract).
- Funzioni di chunking: controllare che un testo di prova sia diviso correttamente.
- Routine di embedding (mock del modello che restituisce vettore fittizio).
- Processi di interrogazione a ChromaDB (usando un mini-database di test).
- Logica di costruzione del prompt (data una domanda, verifica che il prompt includa giusti elementi).
- **Integration Test:** verificano componenti integrati. Esempi:

- Caricamento di un documento di test → chiamata al servizio pipeline (OCR + chunking + indexing) → si può verificare che i chunk siano effettivamente memorizzati in Chroma.
- Chatbot end-to-end: invio di una domanda di prova (input noto) e verifica che la risposta di Claude sia sensata rispetto al materiale di test. Qui si possono usare *mock* o versioni containerizzate di emulazione (ad es. Claude stub).
- Autenticazione: testare il flusso Cognito (signup/login) con credenziali di test e assicurare accesso alle risorse AWS simulate.
- **End-to-End Test (E2E)**: scenari completi in un ambiente di staging. Ad esempio:
 - L'utente si registra e fa login.
 - Carica un PDF di esempio su argomento scolastico.
 - Chatta chiedendo una domanda rilevante. Il sistema deve restituire una risposta coerente (verificato manualmente o via asserzioni su parole chiave).
 - Verificare che la cronologia venga salvata e possa essere recuperata.
 - Verificare che quiz/esercizi generati abbiano senso (anche con difficoltà variabile).
 - Testare flussi di errore: caricamento file non valido, credenziali sbagliate, Bedrock offline.

L'obiettivo dei test è coprire il più possibile le funzionalità critiche: non solo correttezza logica, ma anche resilienza (timeout LLM, risposte nulle) e sicurezza (es. impedire l'accesso a risorse di altri utenti). Automatizzeremo i test con framework come pytest (per unit e integration) e strumenti come Playwright o Selenium per E2E se necessario. Un focus particolare è sulle funzioni di dataflow: ad esempio, unit test per verificare che le funzioni di chunking non producano duplicati o perdite di informazione. In produzione, possiamo usare CI/CD (GitHub Actions) per eseguire i test ad ogni commit.

19. Monitoraggio & Osservabilità (TBD)

Questa sezione è indicativa e verrà sviluppata in futuro. In breve, in una fase successiva si dovrà prevedere un sistema di monitoraggio e logging avanzato. Sarà importante tracciare metriche quali:

- **Obiettivi:** Disponibilità, errori LLM, tassi di successo, latenza end-to-end, utilizzo di risorse (CPU/memoria dei container, token generati).
- **Metriche da monitorare:** numero di documenti caricati per utente, tempo medio di risposta del chatbot, percentuale di risposte corrette (quiz effettuati vs risposte giuste), attacchi o attività sospette sul login.
- **Strumenti consigliati:** AWS CloudWatch per metriche e allarmi, AWS X-Ray per tracciare le chiamate tra microservizi, OpenTelemetry per telemetria custom, Grafana per dashboard (collegandolo a CloudWatch Metrics). Elasticsearch/Opensearch per analisi di log strutturato.

In futuro potremmo aggiungere alert su soglie (es. downtime Bedrock) e analisi automatiche dei log per migliorare la robustezza del sistema.

20. Miglioramenti Futuri

Proton prevede diversi possibili sviluppi successivi:

- **Workflows Agentici:** integrare meccanismi agentici (es. Amazon Bedrock AgentCore) per permettere all'AI di pianificare passi multipli (utile per compiti complessi di risoluzione problemi).
- **Architettura Multi-agente:** design di più agenti specializzati (es. uno dedicato al QA, uno alla generazione di quiz, uno al tutor di matematica).
- **Pianificatore di Studio (Study Planner):** basato sulla progressione dell'utente, un modulo potrebbe suggerire un piano di studio personalizzato usando i contenuti caricati e le domande frequenti.
- **Interfaccia Vocale:** implementare un frontend vocale (e.g. via Alexa o Google Assistant) che consenta agli studenti di parlare invece che digitare. Richiederebbe servizi ASR/TTS (es. Amazon Transcribe, Polly).
- **App Mobile:** una versione dedicata per smartphone (React Native/Flutter) per accesso mobile.
- **Dashboard Analitica:** strumenti per visualizzare le prestazioni di apprendimento dell'utente: punteggi quiz, aree di debolezza, tempo di studio, con analytics su cui basarsi.
- **Miglioramenti RAG/CAG:** includere tecniche avanzate come *Self-RAG* (autocorrezione delle query) o *Multi-Query RAG* per aumentare la completezza delle risposte. Sperimentare CAG con modelli con contesti >200k token.
- **Traduzione e Multilingua:** estendere il sistema per supportare materiale in più lingue (incluso se necessario tradurre via LLM o inserire embed multilingue).
- **Metriche Avanzate:** usare GPT Evaluation per verificare la correttezza delle risposte aperte (metrica automatica di "truthfulness").
- **Integrazione IoT/Edge:** per aule smart o dispositivi educativi che interagiscono con Proton.

21. Conclusione

Proton è progettato come un sistema AI educativo modulare, che sfrutta l'ecosistema AWS e le tecniche RAG/CAG per fornire un tutoring intelligente. Ogni decisione architetturale è motivata da esigenze di semplicità, scalabilità e efficacia didattica. Ad esempio, la scelta di LangChain e ChromaDB rende immediata la creazione di una knowledge base personalizzata **【4†L75-L82】** **【25†L150-L154】** ; l'adozione di container Docker + ECS-Fargate garantisce una pipeline CI/CD efficiente; l'uso di Claude su Bedrock

con ampia finestra di contesto massimizza la qualità delle risposte (fino a centinaia di pagine di input 【2†L66-L74】). Tutte le componenti (storage in S3, processi Textract, pipeline di embedding e retrieval) sono progettate per cooperare in modo fluido e sicuro. La documentazione qui fornita, completa di motivazioni, alternative e dettagli tecnici, è destinata a guidare il team di sviluppo nella realizzazione di Proton da zero, fornendo tutte le informazioni necessarie e le best practice per un'implementazione di successo.

Fonti: Questo documento è basato su ricerche e linee guida AWS e del settore AI, tra cui descrizioni ufficiali di Amazon Bedrock per Claude 【2†L66-L74】 , analisi di pipeline RAG 【4†L75-L82】 【7†L202-L210】 , studi su strategie di chunking 【23†L119-L127】 ed embed 【16†L25-L33】 【18†L171-L179】 , e confronti di database vettoriali 【25†L150-L154】 【32†L576-L584】 .
